System Design, Implementation & System Generation



Today's Topics

- OS Design Goals
- Mechanisms and Policies
- OS Implementation
- Programming Languages
- System Generation
- Booting

Operating System Design



What is OS Design?

Operating System design involves deciding:

- What services the OS should provide
- How components should interact
- How resources should be managed
- How security should be implemented
- How the system should achieve performance and reliability

Important Point

There is **no single perfect OS design**.

Different systems have different requirements.

Goals of OS Design

Operating System design generally considers two types of goals:

User Goals

- Easy to use
- Convenient
- Reliable
- Fast
- Secure

System Goals

- Easy to maintain
- Efficient
- Flexible
- Reliable
- Scalable
- Secure

User Goals vs System Goals

User Goals	System Goals
Easy to use	Easy to maintain
Convenient	Efficient
Fast response	Reliable
Secure	Flexible
User-friendly	Scalable

Policy vs Mechanism

Policy
What should be done?

Mechanism
How should it be done?

Example
Suppose the OS must decide which process gets CPU time.

Policy:
Which process should get CPU?

Mechanism:
How does the scheduler actually perform the selection and context switch?

Policy and Mechanism Example

Security Example
Policy:
Only the administrator can delete a particular system file.

Mechanism:
File permissions and access-control mechanisms enforce that rule.

Remember
Policy = What
Mechanism = How

Operating System Implementation

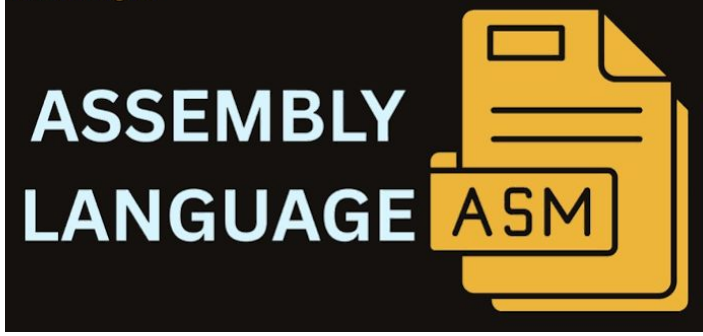
How is an OS implemented?
Operating systems are typically developed using:

- C
- C++
- Assembly language
- Rust in some modern systems/components

Why C?

- Efficient
- Portable
- Low-level hardware access
- Good performance
- Supports system programming

Why Assembly Language?



Assembly language is useful when very low-level hardware control is required.

Used for:

- Boot code
- CPU-specific operations
- Interrupt handling
- Context switching
- Hardware-specific code

Important

Modern operating systems are **not written entirely in assembly language**.
Most large OS codebases use higher-level systems languages,
with assembly used where necessary.

High-Level Language vs Assembly

FEATURE	HIGH-LEVEL LANGUAGE	ASSEMBLY
READABILITY	High	Low
PORTABILITY	Higher	CPU-specific
DEVELOPMENT	Easier	More difficult
HARDWARE CONTROL	Lower-level	Very high
EXAMPLE	C	x86-64 Assembly

Modern OS development balances:
Performance + Hardware Control + Maintainability

Portability

What is Portability?
The ability of software to be adapted to different hardware platforms with relatively little modification.

Example
C code can be compiled for:

- x86-64
- ARM
- RISC-V

with appropriate toolchains and platform-specific code.

Why Important?
Hardware architectures change.
An OS should ideally not need to be completely rewritten for every processor architecture.

OS Implementation Challenges

Operating Systems must handle:

- Concurrency
- Hardware differences
- Memory limitations
- Security
- Performance
- Reliability
- Fault handling
- Scalability

Example

Multiple processes may request the CPU simultaneously.
The OS must handle this safely and efficiently.

Operating System Testing & Debugging

OS development requires extensive testing.

Areas

- Process management
- Memory management
- File systems
- Device drivers
- Networking
- Security
- Hardware compatibility

Why?

A small kernel error can potentially crash the entire system.

What is System Generation?

SYSGEN

System Generation (SYSGEN) is the process of configuring an operating system for a particular hardware environment and system requirements.

It may involve identifying/configuring:

- CPU
- Memory
- Storage devices
- I/O devices
- Device drivers
- Kernel configuration
- Boot parameters

Why is System Generation Needed?

Different computers have different:

- CPUs
- RAM sizes
- Storage devices
- Network interfaces
- Hardware peripherals

Therefore, the OS may need appropriate configuration.

Example

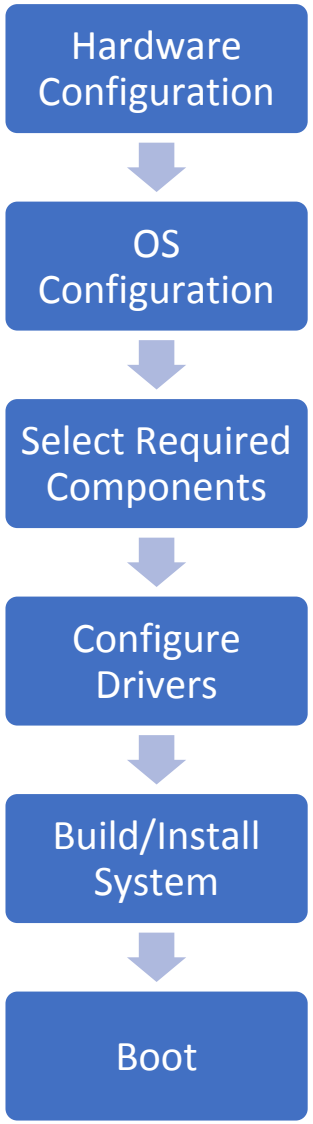
A server may need:

- Network drivers
- Storage drivers
- Server-specific configuration

A small embedded system may need only a limited set of components.

System Generation Process

Simplified process:



What Happens When You Turn ON a Computer?

You press the power button. Windows/macOS/Linux isn’t running yet. So what happens first?

Booting

Definition:
Booting is the process of starting a computer and loading the operating system into memory.

Power ON
↓
Firmware
↓
Bootloader
↓
Operating System Kernel
↓
System Initialization
↓
Login / Desktop

Firmware

Modern computers commonly use:
UEFI (Unified Extensible Firmware Interface)
Firmware runs when the system starts.

Responsibilities include:

- Initialize hardware
- Perform hardware checks
- Identify boot devices
- Start the boot process

Historical Technology
BIOS
Older PCs commonly used BIOS.



Bootloader

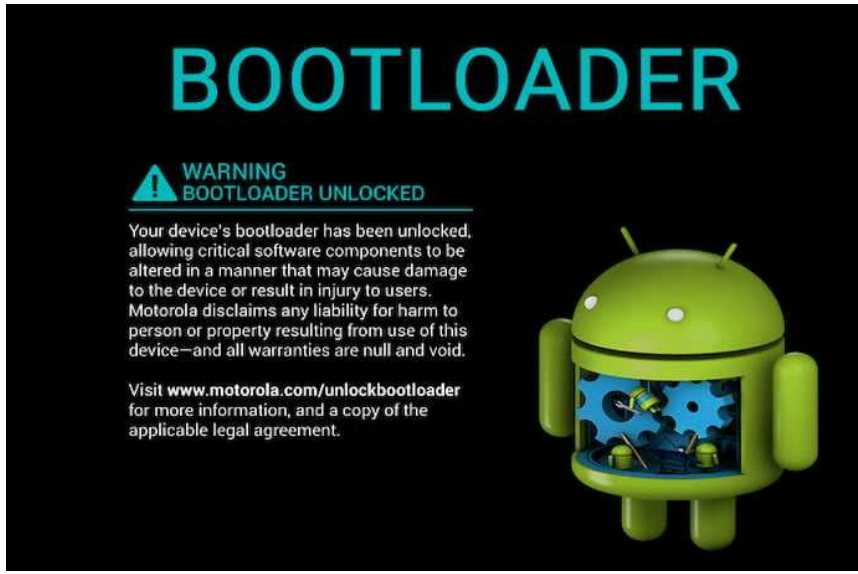
What is a Bootloader?
A **bootloader is software that loads or starts the operating system kernel.**
Simplified Flow

UEFI
↓
Bootloader
↓
Kernel
↓
Operating System

Examples

- GRUB
- Windows Boot Manager
- Apple’s boot process

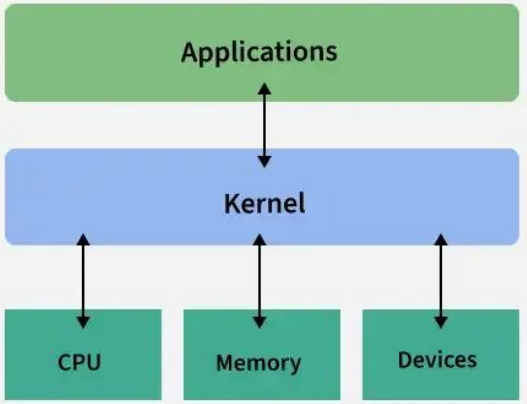




Kernel Loading

After the bootloader starts the kernel:

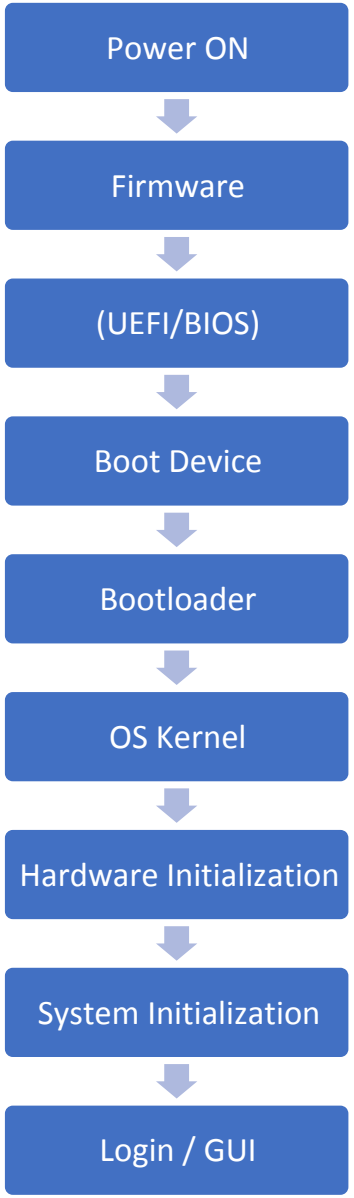
Bootloader
↓
Kernel loaded into RAM
↓
Kernel initializes
↓
Drivers/services initialized
↓
User-space environment starts



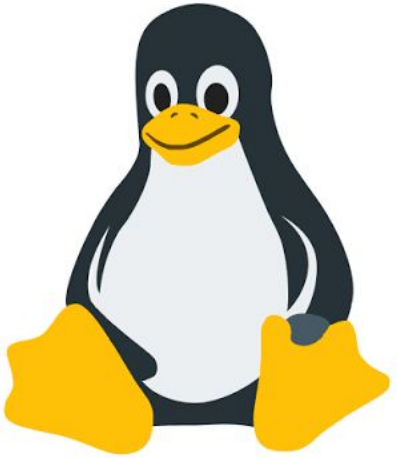
Important
The **kernel is the core component of the operating system** responsible for managing hardware/resources and providing fundamental OS services.

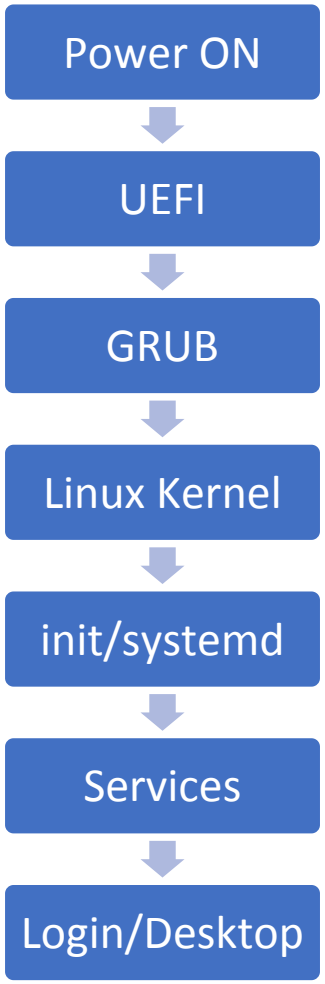
Complete Boot Process

Where does the Operating System kernel come from?
Answer:
It is stored on persistent storage and is loaded into **RAM** during the boot process.

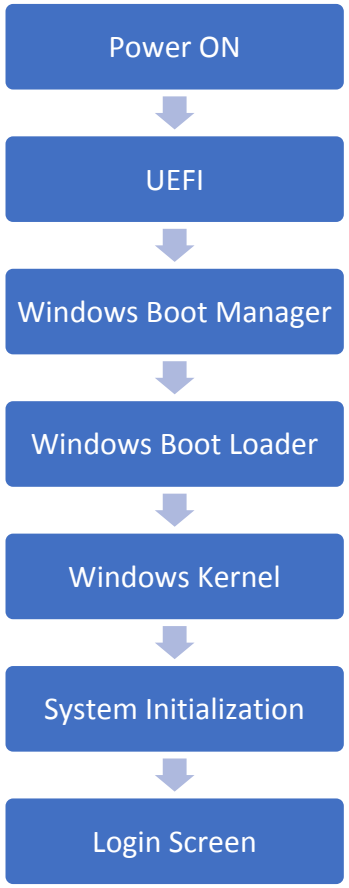


Example: Linux Boot





Example: Windows Boot



Different operating systems have different boot mechanisms, but the overall concept is similar:
Firmware → Bootloader → Kernel → Initialization